

Combinatorial Problem Solving Propositional Satisfiability report

Luis Fernando Chavez Quevedo

June 21, 2026

GitHub repository available at <https://github.com/Luisfc68/combinatorial-problem-solving>

1 Problem summary

The problem considers a set of streets in which some streets are already one-way streets and some others are two-way streets. The goal is to free space that can later be used for parking by converting as many two-way streets as possible into one-way streets. However, this conversion is subject to the following restriction: if, in the original configuration, there exists a path from one intersection to another, then such reachability must be preserved after the modifications. Moreover, the cost of the new shortest path between any such pair of intersections cannot increase by more than a given threshold percentage with respect to the original shortest-path cost. The instance is represented by a cost matrix, where each entry indicates the travel cost from one intersection to another. The solution consists of the selected street orientations, represented as ordered pairs, together with the total number of streets converted into one-way streets.

2 Solving approach

A SAT solver only decides whether a propositional formula in conjunctive normal form is satisfiable; unlike ILP or CP, it has no objective function and no native support for arithmetic or cardinality constraints. Designing a propositional model for the problem from scratch is therefore not trivial at all. Instead of doing so, the approach was to reuse the integer linear programming model from the previous deliverable and translate it into SAT one constraint at a time. The conceptual variables and sets (x , f , the sets O , T , R of one-way arcs, two-way streets and reachable pairs, and the shortest-path matrix D) are kept unchanged; only the language in which each constraint is expressed changes, from linear inequalities to clauses.

This decision had two benefits. First, it made the work considerably simpler: reasoning about how to express a single, already-understood constraint as clauses is much easier than rethinking the whole problem in propositional logic. Second, it serves as a practical test of a hypothesis stated in the ILP report, which was that the flow-based model is the most portable of the three approaches. The ILP report observed that the CP solution relied on a custom propagator that ties it to a specific solver; here the same flow model is carried over to a third, very different paradigm with only the constraint language changing and no solver-specific machinery, which supports that portability claim.

Two aspects of the problem do not map to a single satisfiability query and are handled separately. Since SAT decides feasibility but the problem is an optimization problem, the solver is wrapped in a binary search over the number of conversions k : at each step we ask whether a feasible configuration with at least k conversions exists, and keep the largest feasible k . All feasibility queries are discharged by the `kissat` solver. Furthermore, the course material covers the elementary encodings used as building blocks, but not the complete constructions that the problem requires; where the slides fall short we rely on the literature, as made explicit in the next section and in the bibliography.

3 ILP constraints mappings

We keep the two families of decision variables of the ILP model, now as propositional variables: x_{ij} is true if and only if the arc $i \rightarrow j$ remains active after the changes, and $f(s,t)_{kl}$ is true if and only if the arc $k \rightarrow l$ is used by the path from s to t . Each constraint of the ILP model is translated as follows.

(C1) One-way streets are fixed. For every $(i,j) \in O$ the ILP sets $x_{ij} = 1$; this becomes the unit clause (x_{ij}) .

(C2) Non-existent arcs stay disabled. The ILP sets $x_{ij} = 0$ whenever $t_{ij} = -1$. In the SAT encoding these arcs are simply left out of the model: no flow variable is ever created for them and they appear in no clause, which is equivalent to fixing them to false.

(C3) Two-way streets are not destroyed. The inequality $x_{ij} + x_{ji} \geq 1$ for every $\{i,j\} \in T$ becomes the binary clause $(x_{ij} \vee x_{ji})$.

(C4) Flow uses only active arcs. The inequality $f(s,t)_{kl} \leq x_{kl}$ is the implication $f(s,t)_{kl} \rightarrow x_{kl}$, encoded as the clause $(\neg f(s,t)_{kl} \vee x_{kl})$.

(C5) Flow conservation. The ILP imposes integer flow conservation at every vertex. Over Boolean variables we encode instead the existence of a single unit s - t path by imposing flow conservation [3]: at the source s exactly one outgoing arc carries flow and none enters it; symmetrically at the sink t ; and at every intermediate vertex v at most one incoming and one outgoing arc are used, with an incoming arc used if and only if some outgoing arc is used. The at-most-one sub-constraints use the logarithmic encoding of the course material; the “incoming used $\Rightarrow$ some outgoing used” clauses and their converse are the linking clauses that enforce conservation without a general cardinality encoder. As in the ILP, requiring a single simple path rather than the shortest one is sufficient; any cycle only adds cost (which C6 forbids) so the existence of a within-tolerance walk implies a within-tolerance simple path.

(C6) Time tolerance. The ILP bounds the path cost by $\sum_{(k,l) \in E} t_{kl} f(s,t)_{kl} \leq (1 + \frac{p}{100}) D_{st}$, which is a pseudo-Boolean constraint. We encode it with adder networks [1]. Each term $t_{kl} f(s,t)_{kl}$ is represented as a binary number whose bits are those of t_{kl} with every 1 replaced by the literal $f(s,t)_{kl}$, and the terms are added together with full adders. The slides provide the full adder as a building block and show how to force the output bits of a single adder so as to encode ≤ 0 , but not the construction for a general bound $B = D_{st} + \lfloor p D_{st}/100 \rfloor$. We obtain $\leq B$ by adding the bitwise complement $2^w - 1 - B$ to the sum and forbidding the final carry, while forcing to zero any sum bit above the bit-width of B .

Objective. The ILP maximizes $\sum_{\{i,j\} \in T} (2 - x_{ij} - x_{ji})$. Because SAT only decides feasibility, we introduce for every two-way street a converted indicator c together with the clause $(\neg c \vee \neg x_{ij} \vee \neg x_{ji})$, so that a true c forces the street to be genuinely converted, and we encode “at least k streets converted”, $\sum c \geq k$, with a sorting network: the constraint $\sum x_i \geq k$ is imposed by adding the unit clause y_k on the k -th sorted output. The sorting network itself is Batcher’s odd-even mergesort [2]. The maximum is then found by binary search over k .

4 Implementation details

In contrast with the constraint programming approach, mapping the model to SAT did not require any solver customization, but the propositional encoding does involve a number of specific details worth mentioning:

- **Floyd-Warshall usage.** As in the ILP implementation, the matrix D of original shortest-path costs is computed once at the start with Floyd-Warshall (reused from the checker source code). Here it is used both to compute the tolerance bounds B of C6 and to prune the model.
- **Solving with kissat.** The CNF is written to a temporary file and solved by shelling out to **kissat** through the same pipeline used in the course: the clauses are reversed with **tac**, piped to **kissat**, and the model is recovered by stripping the comment and status lines and keeping the positive literals. An empty output is read as unsatisfiable, which is what the binary search needs.
- **Optimization by binary search.** Since SAT only decides feasibility, the maximum number of conversions is found by binary search over k : each step builds a fresh CNF that additionally forces “at least k conversions” and queries the solver, taking about $\log |T|$ calls per instance.

- **Balanced-tree summation in C6.** The weighted sum of C6 is aggregated as a tree of adders, following the adder-network encoding of [1]: the terms are combined pairwise, as a balanced binary tree, instead of in a linear chain. This keeps the intermediate bit-widths logarithmic and reduces the number of adder instances from $\mathcal{O}(|E|^2)$ to about $\mathcal{O}(|E| \cdot w)$ per pair; on an instance with $n = 11$ it shrank the formula from roughly 3.95M to 464k clauses. The order of summation does not change the constraint, by associativity of addition. This was a necessary change because in the first version some of the instances were exceeding the time limit, so after realizing that, an optimization like this one was researched and included.
- **Overflow-safe comparison.** The $\leq B$ comparison bounds its working width to the bit-length of B and forces every higher sum bit to zero. This avoids an integer overflow in the width computation that, on larger instances, otherwise left those high bits unconstrained and admitted paths above the bound.
- **Avoided creation of unnecessary variables.** As in the ILP, no flow variables are created for unreachable pairs nor for the diagonal. In addition, within a reachable pair (s, t) we skip the flow variable of an arc $k \rightarrow l$ whenever k is unreachable from s or t is unreachable from l in the original graph, since the conversions can only reduce reachability and such an arc can never lie on an $s-t$ path. This roughly halves the encoding on sparse, directed instances and is harmless on dense ones.
- **Logarithmic at-most-one.** The at-most-one constraints of C5 use the logarithmic encoding, introducing $\mathcal{O}(\log n)$ auxiliary variables per constraint instead of the $\mathcal{O}(n^2)$ clauses of the quadratic one. This was also done to reduce the size of the final formula to try to fit all the instances in the time limit.
- **Constant signals.** A single propositional variable is fixed to false with a unit clause to act as the constant 0 (and its negation as 1); it seeds the adders and pads the sorting network.

5 Comparison against CP and ILP

The purpose of this section is to analyze the difference between the results and the performance obtained with the SAT implementation and those of the constraint programming and integer linear programming implementations from the previous deliverables.

The benchmark consists of 111 instances ranging from 4 to 12 crossings. Because the SAT model is a direct translation of the ILP model, it inherits the same asymptotic $\mathcal{O}(n^4)$ growth in flow variables, on top of the auxiliary variables and clauses introduced when the cardinality and pseudo-Boolean constraints are expanded into CNF. The runtime and memory figures reported below should be read with this in mind, and the gap measured on these small instances should only widen on larger ones.

5.1 Results

The output of the SAT solver was compared against the ILP output (and transitively against CP) with `diff -r`. As in the comparison between CP and ILP, the reported objective value (the number of streets converted to one-way) is identical in every instance; the only differences are the chosen orientation of some converted streets, which does not affect optimality, since the problem counts the number of conversions and not their direction. The individual differences are collected in Appendix A.

5.2 Performance

Each solver was executed three times in containers with the same cap on the resources, using the command `docker run -rm -cpuset-cpus="2" -cpuset-mems="0" -memory="8g" -memory-swap="8g" -v ./input:/app/input -v ./out:/app/out <image> time -v /app/solve.sh sdp input out`. The full results can be found in Appendix B; they are summarized in Table 1.

The SAT implementation is clearly the slowest of the three, with a wall-clock time of about 199 s against 24 s for the ILP and 3 s for the CP model. This is the expected outcome of the encoding: the SAT model inherits the same $\mathcal{O}(n^4)$ flow variables as the ILP and, on top of that, expands the pseudo-Boolean tolerance constraint and the cardinality

Metric	CP	ILP	SAT
Wall-clock time (s)	3.07	24.40	199.22
User CPU time (s)	0.68	12.75	153.08
System CPU time (s)	0.72	2.78	44.08
Total CPU time (s)	1.40	15.53	197.16
CPU utilization (%)	45	63	99
Max. resident memory (MB)	7.5	74.5	61.03
Minor page faults	68 653	396 479	1 459 964
Voluntary context switches	3 909	13 804	98 047
Involuntary context switches	76	6 796	84 907

Table 1: Average resource usage over three runs per method.

objective into large networks of adders and comparators, so the formula handed to the solver is considerably larger. Optimization multiplies this cost further, since each instance is solved by binary search and therefore requires about $\log |T|$ independent solver calls, each on a freshly generated CNF.

The remaining figures are consistent with this picture. CPU utilization reaches 99%, against 63% for the ILP and 45% for the CP model, because the SAT pipeline (CNF generation followed by the solver) is almost entirely CPU-bound with little idle time. A noticeable share of the cost is system time, which comes from the solving strategy itself: the whole CNF is written to disk and `kissat` is invoked through a shell pipeline once per binary-search step, multiplying file I/O and process creation; this also accounts for the large number of minor page faults and context switches. Interestingly, despite the much larger formula, the peak resident memory (61 MB) is comparable to, and even slightly below, that of the ILP (74 MB), because the clauses are streamed to disk rather than kept in memory. As with the ILP, the gap relative to CP should only widen on larger instances, given the $\mathcal{O}(n^4)$ growth that both models share.

6 Conclusion

Translating the integer linear programming model into SAT proved to be relatively straightforward (at least easier than solving everything from scratch): each constraint was rewritten as clauses one at a time, with the variables, the sets and the flow formulation carried over unchanged. This supports the hypothesis advanced in the ILP report that the flow-based model is the most portable of the three. Where the CP solution depended on a custom propagator bound to a specific solver, the same model moved to a third, very different paradigm with only the constraint language changing.

Portability, however, comes at a cost in encoding size. The SAT model inherits the $\mathcal{O}(n^4)$ flow variables of the ILP and, on top of that, expands the cardinality and pseudo-Boolean constraints into large networks of adders and comparators, producing a considerably larger formula than the linear program it is derived from.

To summarize, whether SAT is the right tool seems to depend heavily on the model. Porting a model from another paradigm, as we did, keeps the modelling effort low but does not exploit SAT’s strengths and pays for it in performance; a dedicated encoding might recover part of that gap, although the $\mathcal{O}(n^4)$ flow variables are intrinsic to the problem and shared with the ILP, so any improvement would likely be in constant factors rather than in asymptotic growth. It is also the approach for which the course material was least self-sufficient; in the case of constraint programming the reference outside of the slides came from Gecode’s manual, and in the case of ILP once the flow-based approach was selected the rest could be covered with the slides’ content. SAT was the only case where external material had to be extensively reviewed. In any case, for this problem and these instances SAT is the worst of the three options, combining the most intricate encoding with the worst performance.

References

- [1] N. Eén and N. Sörensson. Translating Pseudo-Boolean Constraints into SAT. *Journal on Satisfiability, Boolean Modeling and Computation*, 2006.

- [2] K. E. Batchier. Sorting Networks and Their Applications. In *AFIPS Spring Joint Computing Conference*, volume 32, 1968.
- [3] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice-Hall, 1993.

A Results difference

```
diff -r integer-linear-programming/out/sdp.1.out propositional-satisfiability/out/sdp.1.out
10c10
< 0 2
---
> 2 0
diff -r integer-linear-programming/out/sdp.100.out propositional-satisfiability/out/sdp.100.out
8c8
< 0 3
---
> 3 0
diff -r integer-linear-programming/out/sdp.101.out propositional-satisfiability/out/sdp.101.out
14c14
< 0 5
---
> 5 0
17,18c17,18
< 6 2
< 4 3
---
> 2 6
> 3 4
21c21
< 6 5
---
> 5 6
diff -r integer-linear-programming/out/sdp.102.out propositional-satisfiability/out/sdp.102.out
11c11
< 0 6
---
> 6 0
diff -r integer-linear-programming/out/sdp.103.out propositional-satisfiability/out/sdp.103.out
14,16c14,16
< 8 0
< 2 1
< 1 3
---
> 0 8
> 1 2
> 3 1
19c19
< 5 8
---
> 8 5
diff -r integer-linear-programming/out/sdp.104.out propositional-satisfiability/out/sdp.104.out
12c12
< 3 1
```

```

---
> 1 3
14c14
< 1 6
---
> 6 1
diff -r integer-linear-programming/out/sdp.105.out propositional-satisfiability/out/sdp.105.out
13,16c13,16
< 5 0
< 8 0
< 7 4
< 9 4
---
> 0 5
> 0 8
> 4 7
> 4 9
diff -r integer-linear-programming/out/sdp.106.out propositional-satisfiability/out/sdp.106.out
14c14
< 0 5
---
> 5 0
16c16
< 1 4
---
> 4 1
19,20c19,20
< 3 7
< 4 7
---
> 7 3
> 7 4
diff -r integer-linear-programming/out/sdp.107.out propositional-satisfiability/out/sdp.107.out
16c16
< 6 5
---
> 5 6
diff -r integer-linear-programming/out/sdp.108.out propositional-satisfiability/out/sdp.108.out
14c14
< 2 5
---
> 5 2
diff -r integer-linear-programming/out/sdp.109.out propositional-satisfiability/out/sdp.109.out
15c15
< 0 8
---
> 8 0
17,19c17,19
< 1 10
< 4 2
< 5 2
---
> 10 1
> 2 4

```

```

> 2 5
22c22
< 7 5
---
> 5 7
24,25c24,25
< 9 8
< 10 9
---
> 8 9
> 9 10
diff -r integer-linear-programming/out/sdp.11.out propositional-satisfiability/out/sdp.11.out
10c10
< 3 0
---
> 0 3
13c13
< 3 2
---
> 2 3
diff -r integer-linear-programming/out/sdp.110.out propositional-satisfiability/out/sdp.110.out
15c15
< 0 7
---
> 7 0
19c19
< 2 3
---
> 3 2
23c23
< 4 10
---
> 10 4
diff -r integer-linear-programming/out/sdp.12.out propositional-satisfiability/out/sdp.12.out
11c11
< 4 1
---
> 1 4
diff -r integer-linear-programming/out/sdp.13.out propositional-satisfiability/out/sdp.13.out
10c10
< 0 2
---
> 2 0
12,13c12,13
< 0 5
< 1 4
---
> 5 0
> 4 1
diff -r integer-linear-programming/out/sdp.14.out propositional-satisfiability/out/sdp.14.out
11,12c11,12
< 1 6
< 6 2
---

```

```

> 6 1
> 2 6
diff -r integer-linear-programming/out/sdp.15.out propositional-satisfiability/out/sdp.15.out
7c7
< 0 1
---
> 1 0
diff -r integer-linear-programming/out/sdp.16.out propositional-satisfiability/out/sdp.16.out
9c9
< 0 2
---
> 2 0
11c11
< 3 1
---
> 1 3
diff -r integer-linear-programming/out/sdp.18.out propositional-satisfiability/out/sdp.18.out
10c10
< 2 1
---
> 1 2
12c12
< 2 4
---
> 4 2
diff -r integer-linear-programming/out/sdp.19.out propositional-satisfiability/out/sdp.19.out
7c7
< 0 3
---
> 3 0
diff -r integer-linear-programming/out/sdp.21.out propositional-satisfiability/out/sdp.21.out
10c10
< 6 0
---
> 0 6
diff -r integer-linear-programming/out/sdp.26.out propositional-satisfiability/out/sdp.26.out
10c10
< 5 0
---
> 0 5
diff -r integer-linear-programming/out/sdp.3.out propositional-satisfiability/out/sdp.3.out
9c9
< 1 4
---
> 4 1
diff -r integer-linear-programming/out/sdp.31.out propositional-satisfiability/out/sdp.31.out
9c9
< 1 2
---
> 2 1
diff -r integer-linear-programming/out/sdp.32.out propositional-satisfiability/out/sdp.32.out
11c11
< 4 2
---

```

```

> 2 4
diff -r integer-linear-programming/out/sdp.38.out propositional-satisfiability/out/sdp.38.out
7c7
< 0 2
---
> 2 0
diff -r integer-linear-programming/out/sdp.4.out propositional-satisfiability/out/sdp.4.out
13c13
< 4 1
---
> 1 4
15c15
< 5 4
---
> 4 5
diff -r integer-linear-programming/out/sdp.40.out propositional-satisfiability/out/sdp.40.out
9,10c9,10
< 0 1
< 4 1
---
> 1 0
> 1 4
diff -r integer-linear-programming/out/sdp.41.out propositional-satisfiability/out/sdp.41.out
9c9
< 1 3
---
> 3 1
diff -r integer-linear-programming/out/sdp.43.out propositional-satisfiability/out/sdp.43.out
10c10
< 1 5
---
> 5 1
diff -r integer-linear-programming/out/sdp.5.out propositional-satisfiability/out/sdp.5.out
11c11
< 0 2
---
> 2 0
diff -r integer-linear-programming/out/sdp.52.out propositional-satisfiability/out/sdp.52.out
11c11
< 1 3
---
> 3 1
diff -r integer-linear-programming/out/sdp.55.out propositional-satisfiability/out/sdp.55.out
8c8
< 2 4
---
> 4 2
diff -r integer-linear-programming/out/sdp.57.out propositional-satisfiability/out/sdp.57.out
10c10
< 1 4
---
> 4 1
diff -r integer-linear-programming/out/sdp.59.out propositional-satisfiability/out/sdp.59.out
8c8

```

```

< 1 2
---
> 2 1
diff -r integer-linear-programming/out/sdp.6.out propositional-satisfiability/out/sdp.6.out
8c8
< 0 1
---
> 1 0
diff -r integer-linear-programming/out/sdp.71.out propositional-satisfiability/out/sdp.71.out
13c13
< 5 1
---
> 1 5
15c15
< 3 5
---
> 5 3
diff -r integer-linear-programming/out/sdp.72.out propositional-satisfiability/out/sdp.72.out
10c10
< 2 5
---
> 5 2
diff -r integer-linear-programming/out/sdp.74.out propositional-satisfiability/out/sdp.74.out
11,12c11,12
< 0 4
< 1 2
---
> 4 0
> 2 1
diff -r integer-linear-programming/out/sdp.76.out propositional-satisfiability/out/sdp.76.out
9c9
< 2 3
---
> 3 2
diff -r integer-linear-programming/out/sdp.81.out propositional-satisfiability/out/sdp.81.out
11c11
< 2 5
---
> 5 2
diff -r integer-linear-programming/out/sdp.87.out propositional-satisfiability/out/sdp.87.out
11c11
< 6 5
---
> 5 6
diff -r integer-linear-programming/out/sdp.90.out propositional-satisfiability/out/sdp.90.out
8c8
< 1 4
---
> 4 1
diff -r integer-linear-programming/out/sdp.91.out propositional-satisfiability/out/sdp.91.out
11c11
< 4 0
---
> 0 4

```

```
diff -r integer-linear-programming/out/sdp.93.out propositional-satisfiability/out/sdp.93.out
10c10
< 2 1
---
> 1 2
```

B Performance results

SAT results

```
Command being timed: "/app/solve.sh sdp input out"
  User time (seconds): 149.84
  System time (seconds): 42.15
  Percent of CPU this job got: 99%
  Elapsed (wall clock) time (h:mm:ss or m:ss): 3:13.69
  Average shared text size (kbytes): 0
  Average unshared data size (kbytes): 0
  Average stack size (kbytes): 0
  Average total size (kbytes): 0
  Maximum resident set size (kbytes): 62520
  Average resident set size (kbytes): 0
  Major (requiring I/O) page faults: 0
  Minor (reclaiming a frame) page faults: 1460128
  Voluntary context switches: 96031
  Involuntary context switches: 83166
  Swaps: 0
  File system inputs: 0
  File system outputs: 1284160
  Socket messages sent: 0
  Socket messages received: 0
  Signals delivered: 0
  Page size (bytes): 4096
  Exit status: 0
```

```
Command being timed: "/app/solve.sh sdp input out"
  User time (seconds): 171.48
  System time (seconds): 49.25
  Percent of CPU this job got: 99%
  Elapsed (wall clock) time (h:mm:ss or m:ss): 3:42.83
  Average shared text size (kbytes): 0
  Average unshared data size (kbytes): 0
  Average stack size (kbytes): 0
  Average total size (kbytes): 0
  Maximum resident set size (kbytes): 62436
  Average resident set size (kbytes): 0
  Major (requiring I/O) page faults: 104
  Minor (reclaiming a frame) page faults: 1459417
  Voluntary context switches: 98165
  Involuntary context switches: 84979
  Swaps: 0
```

File system inputs: 13520
File system outputs: 1284168
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

Command being timed: "/app/solve.sh sdp input out"
User time (seconds): 137.91
System time (seconds): 40.84
Percent of CPU this job got: 98%
Elapsed (wall clock) time (h:mm:ss or m:ss): 3:01.14
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 62516
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 82
Minor (reclaiming a frame) page faults: 1460348
Voluntary context switches: 99945
Involuntary context switches: 86578
Swaps: 0
File system inputs: 10160
File system outputs: 1284176
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

ILP results

Command being timed: "/app/solve.sh sdp input out"
User time (seconds): 12.69
System time (seconds): 2.79
Percent of CPU this job got: 63%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:24.45
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 76156
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 185
Minor (reclaiming a frame) page faults: 396162
Voluntary context switches: 13853
Involuntary context switches: 6707
Swaps: 0
File system inputs: 31648
File system outputs: 0

Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

Command being timed: "/app/solve.sh sdp input out"
User time (seconds): 12.63
System time (seconds): 2.79
Percent of CPU this job got: 63%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:24.18
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 75776
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 396262
Voluntary context switches: 13776
Involuntary context switches: 6790
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

Command being timed: "/app/solve.sh sdp input out"
User time (seconds): 12.92
System time (seconds): 2.76
Percent of CPU this job got: 63%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:24.58
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 77056
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 397013
Voluntary context switches: 13782
Involuntary context switches: 6892
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0

Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

CP results

Command being timed: "/app/solve.sh sdp input out"
User time (seconds): 0.69
System time (seconds): 0.74
Percent of CPU this job got: 46%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:03.10
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 7680
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 69153
Voluntary context switches: 3947
Involuntary context switches: 83
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

Command being timed: "/app/solve.sh sdp input out"
User time (seconds): 0.66
System time (seconds): 0.72
Percent of CPU this job got: 45%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:03.04
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 7680
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 68951
Voluntary context switches: 3947
Involuntary context switches: 68
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

Command being timed: "/app/solve.sh sdp input out"
User time (seconds): 0.69
System time (seconds): 0.71
Percent of CPU this job got: 45%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:03.08
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 7680
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 67854
Voluntary context switches: 3834
Involuntary context switches: 76
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0